

BioPrint — Behaviour-Based Login Security

Track: Event 2 — BioPrint: Behaviour-Based Login Security · Team submission report

1. What BioPrint does

A password proves that someone knows a secret. It does not prove who is at the keyboard. BioPrint checks the second thing.

It watches an ordinary username and password form from the moment the page loads: the rhythm of the typing, the path the mouse takes to the button, the click, and the pauses in between. The host application checks the password through its own callback, and the password is removed from the request before any behavioural code runs.

A login ends in one of three ways, and none of them is a code sent to a phone.

Outcome	When	What the user sees
Accept	correct password, behaviour matches the enrolled profile, on a known device, with a complete recording	signed in, after a few milliseconds of server time
Step up	correct password, but the behaviour is uncertain: a new device, a thin recording, or a score below the person's own threshold	one ten-second check — type a short phrase, click three targets. The result is accept or reject, never another step up
Reject	a replayed or scripted attempt, a clear mismatch, or the wrong password	blocked, with a written explanation of which signals failed

Two questions are kept apart from start to finish. **Identity** asks whether this matches the person who enrolled. **Fraud** asks whether a hand produced this at all. We never average them into one number, because a low identity score means "ask again" while a failed fraud check means "this was not a person". A script that reproduces someone's timing perfectly is still blocked, because the block does not depend on the timing being wrong.

The full sequence is: activity recorded before anyone signs in (optional, and the code calls it *ambient*) → the login → the step-up check, only when needed → monitoring afterwards. Monitoring can only shorten a session. Three mismatches in a row, or five stale minutes, force a fresh login; nothing observed after sign-in extends it.

2. The fingerprint

Every recording is reduced to a fixed set of measurements, grouped by what produced them.

Group	What we measure
Username typing	hold time per key, gap between keys, release-to-press gap, overlap, corrections, pauses and their spread, burst length, Shift use, and the hold time of each letter and the latency of each letter pair. The username is not a secret, so we can name the keys
Password typing	the same timings, from position only. The recorder keeps the slot and the category of each key press, never the key, so the model learns how this person types their fourth character without knowing what it is
Mouse movement	speed and entry speed divided by the straight-line distance to the target, so distance is not mistaken for identity; acceleration, straightness, turning angle, curvature, overshoot, correction rate, approach angle
Click	how long the button is held, how long the pointer hovers first, where on the button it lands
Timing between the two	the pause between username and password, the gap between the last keystroke and the first mouse movement, travel time, and whether the person used Tab or the mouse

That last group matters more than it looks: the pause between the keyboard and the mouse is a habit, hard to fake deliberately, and in the simulation it separates people better than the mouse path does.

Building the profile. For each measurement the enrolment rounds give a centre and a spread. The centre is the median. The spread is $1.4826 \times$ the median absolute deviation, which puts it on the same scale as a standard deviation for normally distributed data, with a floor underneath — 15 ms for a key hold — so that an unusually consistent person does not get an impossibly narrow band. Each measurement also carries a weight, the floor divided by the spread, clamped to [0.25, 1], so noisy measurements count for less. We use medians rather than means because one distracted round must not move the model: across eight to ten rounds a single outlier drags a mean noticeably and barely moves a median.

3. The formula

One measurement. For each measurement x , with centre c and spread s :

$$z = |x - c| / s, \text{ capped at } 4$$
$$\text{score} = \exp(-z^2 / 2)$$

A value on the enrolled centre scores 1. One spread away scores 0.61, two away 0.14, and past four spreads the score stops falling, so one wild value cannot drive a group to zero on its own.

Combining them. Each group is the weighted geometric mean of its measurements. The identity score is the geometric mean over the five groups, with fixed weights — username 0.2, password 0.3, mouse 0.25, click 0.1, timing between the two 0.15 — each multiplied by how much usable data that group actually carried.

Using a geometric mean at both levels is the most consequential choice in the system. An average lets one strong group carry a weak one: a mouse path that matches would paper over a password rhythm that does not. A geometric mean cannot. Any group scoring near zero pulls the result near zero, which is what we want, because an impostor who has the password only has to fail one group to be caught. Groups with no data — autofill, or a keyboard-only submit — drop out of the mean and reduce the quality figure. They never raise the confidence of what is left.

Where the threshold comes from. A geometric mean of this many terms has no fixed scale. The more measurements a group carries, the lower everyone scores, including the genuine owner. Real typists on the CMU data score 0.15 to 0.6 on a thirty-measurement field; scores above 0.85 are what our synthetic test fixtures produce, not what people produce. So a fixed threshold is meaningless, and both thresholds come from the person's own enrolment. Each enrolment round is scored against a model built from the other rounds only — leave-one-out — which shows how much this person varies from themselves.

Threshold	Rule	Meaning
Accept	10th percentile of the leave-one-out scores, clamped to [0.40, 0.90]	at least as close as your own worst enrolment round
Reject	$\min(0.4 \times p_{10}, \text{accept} - 0.1)$	far below anything your own rounds scored
Step-up check	10th percentile, clamped to [0.65, 0.90], on its own model	the same rule, on its own scale

Above the accept threshold the login goes through. Below the reject threshold, and only with a complete recording on a known device, it is blocked on behaviour alone. In between, the step-up check decides. The final decision weights the two stages 0.35 for the login and 0.65 for the check, so the check dominates and a passing login score cannot rescue a failed check.

We arrived at this rule by measuring. Each row is the same engine with one line changed, over 30 simulated people and 51 real CMU typists. The numbers are genuine logins pushed to a step up, and impostors let through.

Accept rule	Simulation	CMU, 10 rounds	CMU, 200 rounds
$\max(0.85, \text{median} - 3 \cdot \text{MAD})$ (our first attempt)	96.5% / 0.0%	60.8% / 1.6%	59.0% / 3.9%
10th percentile, no clamp	4.5% / 7.9%	19.1% / 7.9%	8.2% / 11.7%
Fixed 0.5	29.5% / 0.5%	41.5% / 0.4%	26.0% / 2.9%
10th percentile, clamped [0.40, 0.90] (current)	6.8% / 2.9%	30.4% / 1.1%	13.4% / 6.6%

The first rule worked on our fixtures and was close to useless on people: it stepped up almost every genuine login. The 0.40 floor exists because a thirty-measurement group has leave-one-out minima near 0.15, where 18% of impostors walk through.

4. The four checks

Every stage runs the same four checks, and each can fail on its own.

Check	Question	Passes when
Identity	does this match this person?	score at or above their accept threshold
Evidence quality	is there enough to judge?	quality at or above 0.75 on a login, 1.0 on a check, recording complete
Humanity	did a hand produce this?	human score at or above 0.8 on a login, 0.6 on a check, no automation rule fired
Freshness	is this a new performance?	the one-time challenge was consumed, and this is not a repeat of a recent recording

Accepting without a step up also requires a device the account has used before; a new browser always gets the check.

The automation and replay rules are written so a person can read them back, and they do not look at identity at all. Gaps between keystrokes varying by less than a millisecond, or falling into a few repeated values: human typing does not do that, a timer does. Impossible speed, a key held under 8 ms. A mouse path that is mathematically straight *and* sampled on a perfect clock; either alone is weak, since a trackpad flick is nearly straight, but together they are decisive. Events the browser marks as script-generated: a handful is only a hint, because extensions generate events, but every event being synthetic is not. No pause between the last keystroke and the first mouse movement, because moving a hand takes time. And replay: an exact repeat is caught by a SHA-256 hash over the events with timestamps shifted to start at zero, a near repeat by comparing normalised timings and the mouse path against the account's last 128 recordings, which catches a capture replayed 3% slower.

The explanation. Every decision carries a headline, a summary, one line per group and per check, and the fraud signals. It reads back numbers the scoring already produced and decides nothing: *"Password inter-key interval was 52% longer than the enrolled median (243 ms vs 160 ms, 2.8 spreads off)."* Decision time is reported on every result, usually 1–5 ms. Turning the detail off is one flag, since the centres and spreads are the profile itself.

5. What we built on

None of the mathematics is new. Each piece was chosen because published work already showed it solves a specific problem, which is how we got to a working system in a week.

Robust per-user statistics. Killourhy and Maxion (2009) compared fourteen detectors on one keystroke corpus and found the *scaled Manhattan* family best: each measurement's deviation divided by its own spread, which is exactly our z . Monroe and Rubin (2000) had already argued for medians over means here. Our benchmark re-implements their detector in the same file as ours and reproduces their published 9.6% error rate to three decimals, which is how we know the harness is not flattering us.

Mouse dynamics. Gamboa and Fred (2004) and Ahmed and Traore (2007) established speed, acceleration, straightness and click timing over normalised segments. Zheng, Paloski and Wang (2011) showed that angle measurements — turning angle, curvature, approach angle — stay stable between sessions and are hard to imitate, because they describe the shape of a movement rather than its speed. That is why our mouse features lean on angles, and why we divide speed by the distance to the target.

Dynamic time warping. Sections 2 and 3 reduce a recording to medians and spreads, which throws away order: two people with identical median hold times can move through a phrase completely differently. Dynamic time warping (Sakoe and Chiba, 1978) compares two sequences directly while letting them stretch against each other, so a person typing the same phrase 10% faster still matches themselves, where a point-by-point distance would call them a stranger. We constrain the alignment to a band of eight steps, the restriction from the original paper; without it the algorithm can stretch a slow recording onto a fast one and declare anything similar to anything.

We use it only on the step-up check, where the phrase and targets are known and two recordings are comparable. Up to 24 recordings are kept from enrolment, balanced across phrases, and an attempt is compared only against recordings of the same phrase on the same class of device. Distances become scores, combine across typing, mouse, curvature and touch, and the median of the best three is combined with the statistical score and the letter-pair timings.

And it is switched off. It runs on every step-up check and its score is recorded, but it does not affect any decision. Turning it on requires a report from recorded sessions with real participants, described in `docs/EVALUATION.md`: at least two enrolled users, all eight attack categories in a held-out test set, and error rates no worse than the current matcher with at least one strictly better. The only data we have at that volume is synthetic, and synthetic data cannot show that a matcher works on people.

Continuous authentication. Bours (2012) treats trust as something each sample spends down rather than something granted once at the door. That is the reasoning behind monitoring being able to shorten a session and never extend it. What we deliberately did not use is a trained classifier. One would need hundreds of samples per person and a population of impostors to train against. We need to enrol someone in two minutes, on a laptop.

6. What we built

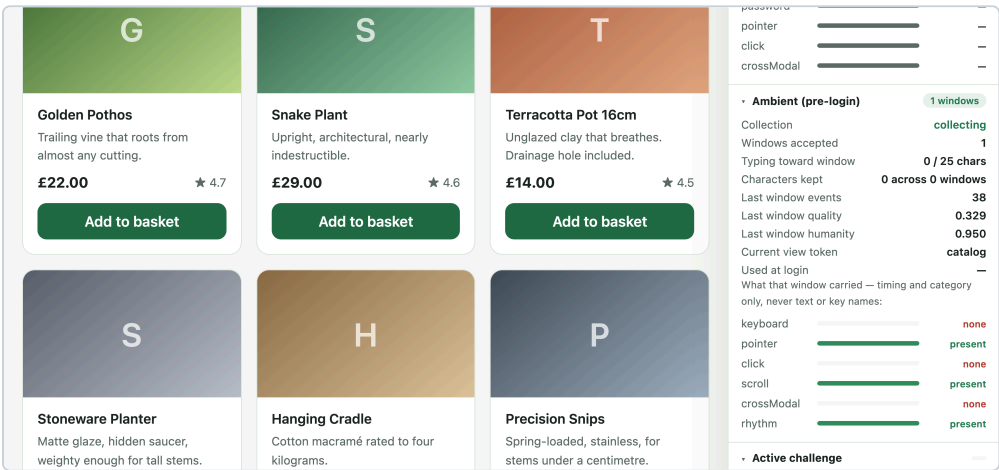
The server. Node 24 and its built-in modules only — HTTP, SQLite, crypto, the test runner. No dependencies, so there is nothing to install and nothing to audit. Every request carrying behaviour is answered against a single-use challenge tied to the account, the purpose, the device and an expiry, and the session cookie is HTTP-only and scoped to the mount path.

The SDK. The server is a handler you mount inside an existing application:

```
app.use(middleware(core, {basePath: '/bioprint', verifyPassword, authorizeEnrollment, onDecision}));
```

Three callbacks are the entire surface the host implements. `verifyPassword` checks the credential, `authorizeEnrollment` decides whether a browser may train an account, and `onDecision` is awaited before the response is sent, so the host sees the outcome first. That last point is the one we would defend hardest: **an accepted decision is not a signed-in user**. BioPrint never creates a session for the host. Without a `verifyPassword` callback every password fails, so a misconfigured mount fails closed. On the browser side there is one entry point per stage, with TypeScript definitions, and unmatched paths fall through to the host application.

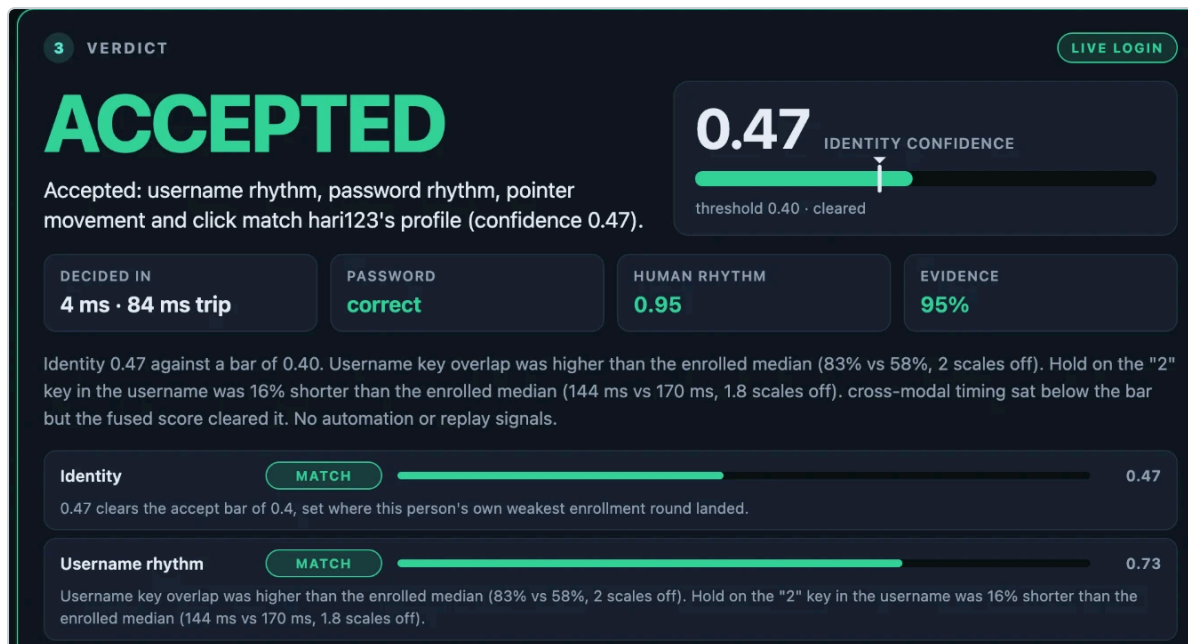
The shop. `examples/shop` is a small store that mounts BioPrint properly, and it exists to show this is a component rather than a demo. It has no backend of its own — catalogue and basket live in the browser — but it has the two halves of a real deployment: an account system it owns, and a mount point.



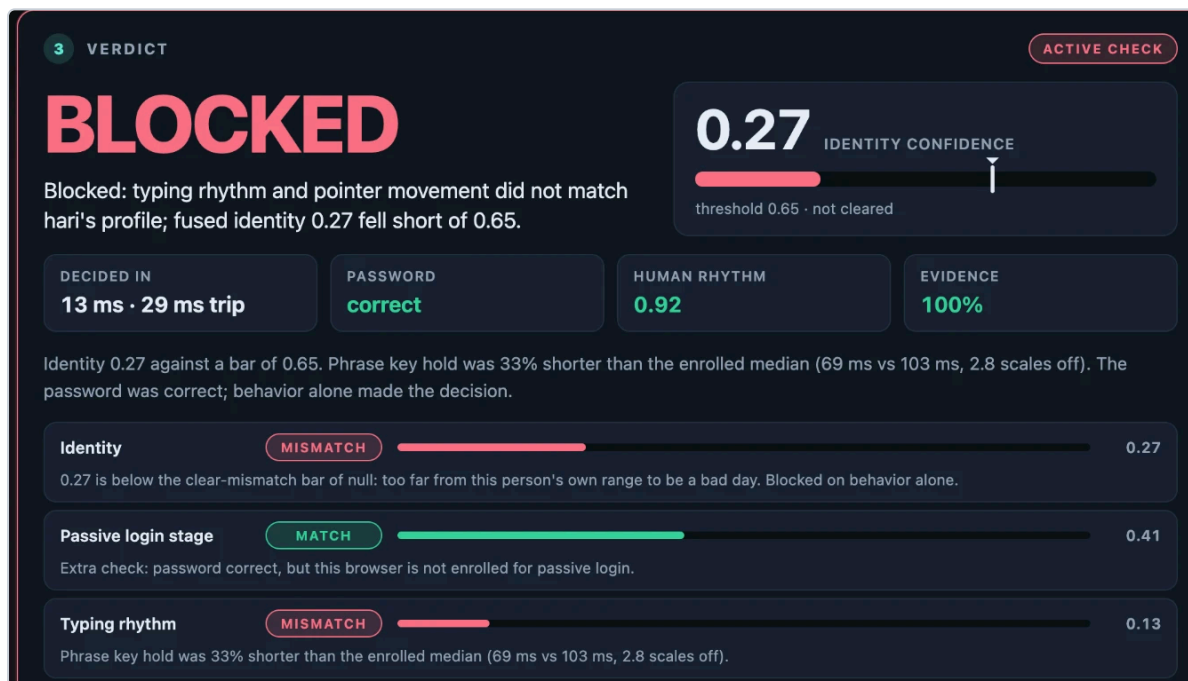
Browsing the shop before signing in. The inspector on the right shows one window of activity accepted, what it carried — pointer, scroll and rhythm present, keyboard and click absent — and the line that matters for privacy: timing and category only, never text or key names.

Browsing it shows activity being recorded before anyone has claimed an identity. That evidence can only withhold confidence, never supply it, since at that point it is attacker-controlled. Signing in either goes straight through or raises the step-up check in a modal, and either way the `shop`'s cookie is minted in `onDecision`. The enrolment key is generated at boot and never reaches the page. If an account has no profile yet the SDK says so and lets the host choose; this shop signs in on the password, marks the session unverified, and refuses that path once a profile exists, so the fallback cannot be used to walk around the behavioural check.

The demo. A login page that shows the whole system working. Enrolment is the user's own login repeated: eight natural logins for the login profile, six phrase-and-target rounds for the step-up profile, with the server reporting when it has enough and rounds advancing on their own. After that, every attempt lands in the verdict panel: the outcome, the confidence against that person's own threshold, the decision time, and one line per signal.



The account holder logging in. Identity 0.47 against their own accept bar of 0.40, decided in 4 ms, with every group matching and each one quantified against the enrolled medians. The score is not near 1, and it is not meant to be: 0.47 is a normal score for a real person across five groups, which is the whole reason the bar comes from their own enrolment rather than a fixed number.



The same page blocking a login on behaviour alone. The password for the `hari` account was correct, but the typing rhythm and pointer movement did not match its profile, so the attempt went to the step-up check and failed it: identity 0.27 against that account's bar of 0.65. Humanity (0.92) and freshness both pass, so this was a person — just not the right one, and knowing the password did not save it.

Four things are worth showing in three minutes: a genuine login accepted; a teammate typing the correct password and being stepped up or blocked; a scripted login through the same recording path, rejected as automation; and a replay of a captured login, rejected as a repeat.

7. Evidence

Three commands, all reproducible from the repository with no network access.

Real human data. `npm run benchmark` runs our login matcher over Killourhy and Maxion's dataset: 51 people, 400 typed password entries each, 20,400 in total, across eight sessions at least a day apart. Each repetition is rebuilt into the exact event stream a browser would emit and pushed through the real matcher, not a re-implementation of it. Lower is better.

Matcher	Enrolment	Mean equal error rate
Scaled Manhattan, the paper's best detector, as published	200 repetitions	9.6%
Scaled Manhattan, our re-implementation	200 repetitions	9.6%
BioPrint	200 repetitions	8.2%
Scaled Manhattan	10 repetitions	8.1%
BioPrint	10 repetitions	6.0%

Two findings we would rather state than bury: enrolling on the first ten passwords someone ever types gives 27%, because people speed up sharply while learning a new string, and holding out test data from the same sessions as the training data changes the result by about a point, so drift is not what stands between this and a perfect matcher.

The whole system end to end. `npm run simulate` drives the real engine, SQLite on disk, with 30 generated people and four classes of attack. This is synthetic: it shows what the engine does when fed plausible behaviour and plausible attacks, not how people behave.

Attempt	n	Accept	Step up	Reject
The genuine owner, new session	600	93.2%	6.8%	0.0%
Another person with the correct password	1740	2.9%	52.1%	45.1%
Someone moved 50% toward the victim	360	50.3%	47.8%	1.9%
Scripted, exact timing, straight mouse path	180	0.0%	0.0%	100.0%
Scripted, randomised timing	90	0.0%	17.8%	82.2%
A captured login replayed exactly	30	0.0%	0.0%	100.0%
The same capture, 3% slower	30	0.0%	0.0%	100.0%

No scripted or replayed attempt reached accept in any run. The owner is accepted outright in 93% of logins and asked for the ten-second check in the rest, never blocked. Someone with the password but not the habits is blocked outright 45% of the time and challenged otherwise. The 50% mimic is the honest weak spot: half get through, which is what we would expect of an attacker halfway to the victim on every measurement.

Server decision time over 2,970 logins: median 3.2 ms, 95th percentile 4.5 ms, worst case 11.9 ms, including the SQLite write. `npm test` runs 29 tests over the protocol, the scoring, the automation and replay rules and the explanation. All pass.

8. Limits and privacy

The thresholds come from a person's own enrolment, so they measure how much that person varies from themselves, not how often an impostor gets through. The clamps carry that second piece of evidence, from the CMU data and the simulation, but they are clamps and not a model of attackers.

The CMU corpus is keystrokes only. Real error rates for the full five-group system need recorded sessions with real participants on separate days, which we do not have. That is why dynamic time warping and profile adaptation are both written, both tested and both switched off.

Password keys, codes and values are rejected by an allowlist before anything is stored, so the password is not in the database in any form. Username letter pairs are kept, since the username is not a secret. The SQLite file is not encrypted on disk; there is a tool for encrypted snapshots and for applying a retention window, and the repository's data directory should not be shared once real people have enrolled in it.

References. Killourhy & Maxion 2009 (DSN); Monroe & Rubin 2000 (Future Generation Computer Systems 16:4); Gamboa & Fred 2004 (SPIE 5404); Ahmed & Traore 2007 (IEEE TDSC 4:3); Zheng, Paloski & Wang 2011 (ACM CCS); Sakoe & Chiba 1978 (IEEE TASSP 26:1); Bours 2012 (Information Security Technical Report 17:1-2).